
CS771 Minor Assignment 2

Diya Patel diyahp24@iitk.ac.in
Saurya Singh sauryas24@iitk.ac.in
Medini Mudraboina medinim24@iitk.ac.in
Shruti Sahu shrutisahu24@iitk.ac.in
Vasu Goyal vasug24@iitk.ac.in
Deepak Kumar deepakk24@iitk.ac.in

1 Question 1

Our main objective is

$$\arg \max_{c,v} \mathbb{P}[y = c, v \mid x^o]$$

We can divide the probability into two terms as shown:

$$\arg \max_{c,v} \mathbb{P}[y = c \mid x^o] \mathbb{P}[v \mid y = c, x^o]$$

First we maximize with respect to v , note that the first term is not dependent on v , so our problem now boils down to

$$\arg \max_v \mathbb{P}[v \mid y = c, x^o]$$

Let the Gaussian for the class c be $\mathcal{N}(\mu^c, \Sigma^c)$. According to the given hint, we have:

$$\arg \max_v \mathcal{N}(v \mid \bar{\mu}^c, \bar{\Sigma}^c)$$

Where

$$\begin{aligned} \bar{\mu}^c &= \mu^{m,c} + \Sigma_{mo}^c (\Sigma_{oo}^c)^{-1} (x^o - \mu^{o,c}) \\ \bar{\Sigma}^c &= \Sigma_{mm}^c - \Sigma_{mo}^c (\Sigma_{oo}^c)^{-1} \Sigma_{om}^c \end{aligned}$$

For a Gaussian, the maximum value of the function occurs at the mean. Hence the solution for this maximization problem is $x_m = \bar{\mu}^c \implies \mathcal{N}(\bar{\mu}^c \mid \bar{\mu}^c, \bar{\Sigma}^c) = \frac{1}{\sqrt{(2\pi)^{d_m} |\bar{\Sigma}^c|}}}$, where d_m is the number of missing pixels.

Now we maximize the original expression with respect to c , after substituting the value of the second term. The problem becomes

$$\arg \max_c \mathbb{P}[y = c \mid x^o] \frac{1}{\sqrt{(2\pi)^{d_m} |\bar{\Sigma}^c|}}$$

Remove all terms independent of c and we get our final answer as

$$\hat{y}_{single} = \arg \max_c \frac{\mathbb{P}[y = c \mid x^o]}{\sqrt{|\bar{\Sigma}^c|}}$$

Once we have this, we can simply write the missing pixels as $\hat{x}^m = \bar{\mu}_{\hat{y}_{single}}$. Now to calculate the value of c for which the expression is maximum, we change our problem a bit and instead find

$$\hat{y}_{single} = \arg \max_c \log \left(\frac{\mathbb{P}[y = c \mid x^o]}{\sqrt{|\bar{\Sigma}^c|}} \right)$$

We split the probability into, $\mathbb{P}[y = c \mid x^o] \propto \mathbb{P}[y = c] \mathbb{P}[x^o \mid y = c]$ (the denominator $\mathbb{P}[x^o]$ does not depend on c and thus drops out of the arg max)

For the MNIST data set we have $c = \{0, 1, \dots, 9\}$, We will calculate $\mathbb{P}[y = c]$ based on how many

data points each class in our training data set has, and for the second term, because the full image was modeled as a Gaussian, this will also be modeled as a Gaussian so,

$$\mathbb{P}[x^o \mid y = c] = \mathcal{N}(x^o \mid \mu^{oc}, \Sigma_{oo}^c)$$

Then we calculate the value of $\sqrt{|\bar{\Sigma}^c|}$, and find the value of this function for each label c , and then pick the maxima from it.

In the two step inference procedure, the final ans is

$$\hat{y}_{two} = \arg \max_c \mathbb{P}[y = c \mid x^o]$$

and

$$\hat{x}^m = \bar{\mu}_{\hat{y}_{two}}$$

and as we've seen above, for the single step inference procedure, the final ans is

$$\hat{y}_{single} = \arg \max_c \frac{\mathbb{P}[y = c \mid x^o]}{\sqrt{|\bar{\Sigma}^c|}}$$

and

$$\hat{x}^m = \bar{\mu}_{\hat{y}_{single}}$$

We can see that the two step and single step procedures are not identical. The two step score, for each class c is solely $\mathbb{P}[y = c \mid x^o]$ whereas the single-step score additionally divides by $|\bar{\Sigma}^c|$.

The term $|\bar{\Sigma}^c| = |\Sigma_{mm}^c - \Sigma_{mo}^c(\Sigma_{oo}^c)^{-1}\Sigma_{om}^c|$ is the determinant of the conditional covariance of the missing pixels, given the observed pixels under class c . It is a measure of how spread out (uncertain) the missing pixel distribution is for that class. Classes with a large $|\bar{\Sigma}^c|$ are penalised by the single step method relative to the two-step method. The two procedures coincide only in the case where $\bar{\Sigma}^c$ is identical for all classes.

2 Question 2: Code Modifications for Single-Step Inference

Overview of Modifications

The lecture code separates prediction into two functions: `predictClassScores`, which computes the log-score $\log \mathbb{P}[x^o \mid y = c] + \log \mathbb{P}[y = c]$ for a single class, and `predictGen`, which calls it for every class and returns the argmax. A third function, `reconstructImages`, fills in the missing pixels using the Gaussian conditional mean once a label has been predicted.

From Question 1, the single-step score for class c is

$$s_c(x^o) = \underbrace{\log \mathbb{P}[x^o \mid y = c] + \log \mathbb{P}[y = c]}_{\text{original two-step score}} - \frac{1}{2} \log |\bar{\Sigma}^c|, \quad (1)$$

where $\bar{\Sigma}^c$ is the Schur complement as defined in Question 1

The required modifications are:

1. **Merge `predictClassScores` and `predictGen` into a single function `predictGenSingleStep`.** The extra term $-\frac{1}{2} \log |\bar{\Sigma}^c|$ is computed *inside* the same loop that already computes the two-step score, so no additional pass over the model is needed.
2. **Compute $\bar{\Sigma}^c$ for each class using the already-available covariance matrix Σ^c .** This requires slicing Σ^c into blocks according to the observed/missing pixel mask: Σ_{oo}^c (observed $\times$ observed), Σ_{mm}^c (missing $\times$ missing), and Σ_{mo}^c (missing $\times$ observed). Note that $(\Sigma_{oo}^c)^{-1}$ is already computed for Term 1 and can be reused directly for the Schur complement, avoiding redundant computation.
3. **Use `numpy.linalg.pinv` (pseudoinverse) for $(\Sigma_{oo}^c)^{-1}$,** consistent with the existing lecture code. MNIST images have many border pixels that are always white across all training examples; their variance is exactly zero, making Σ_{oo}^c rank-deficient. `numpy.linalg.inv` would raise an error, whereas `numpy.linalg.pinv` handles this by treating them as having infinite inverse variance (i.e. zero contribution).

4. Use `numpy.linalg.slogdet` for $\log |\bar{\Sigma}^c|$, again consistent with the lecture code. If the sign is non-positive, $\log |\bar{\Sigma}^c|$ is set to 0 as a safe fallback, mirroring the same convention used in the lecture code for $\log |\Sigma_{oo}^c|$.
5. `reconstructImages` is **left unchanged**. Both methods produce $\hat{x}^m = \bar{\mu}_{\hat{y}}$, so the reconstruction step is identical regardless of which prediction rule was used to obtain $\hat{y}$.

Modified Python Code

The complete replacement for `predictClassScores` and `predictGen` is given below as a single merged function.

```
def predictGenSingleStep( X, model, C, mask = [] ):
    # If no mask was provided, consider all coordinates of the feature vector
    if len(mask) == 0:
        mask = np.ones( (X.shape[1],), dtype=bool )

    classScores = np.zeros( (X.shape[0], C) )
    for mu, Sigma, c, p in model:
        XCent = X[:,mask] - mu[mask]

        SInv = lin.pinv( Sigma[mask,:][:,mask] ) #using pseudoinverse

        (sign, logdet) = lin.slogdet( Sigma[mask,:][:,mask] )

        if sign <= 0:
            SLogDet = 0
        else:
            SLogDet = logdet

        # --- Term 1: log P[x~o | y=c] (identical to original code) ---
        term1 = 0.5 * (-SLogDet - np.sum( np.multiply( np.dot(XCent, SInv), XCent), axis = 1 ))

        # --- Term 2: log P[y=c] ---
        term2 = np.log( p )

        # --- Term 3 (NEW): -0.5 * log|Sigma_bar_c| ---
        SigmaBar = Sigma[mask,:][:,~mask] - \
            Sigma[mask,:][:,mask].dot( SInv ).dot( Sigma[mask,:][:,~mask] )
        (sign_bar, logdet_bar) = lin.slogdet( SigmaBar )
        if sign_bar <= 0:
            SBarLogDet = 0
        else:
            SBarLogDet = logdet_bar
        term3 = -0.5 * SBarLogDet
        classScores[:,c] = term1 + term2 + term3
    return np.argmax( classScores, axis = 1 )
```

The function is used as a drop-in replacement for `predictGen`:

```
# Two-step (original code):
yPred_two = predictGen(XTestCensorFlat, model, C, mask)

# Single-step (new):
yPred_single = predictGenSingleStep(XTestCensorFlat, model, C, mask)

# Reconstruction is identical for both methods:
XRecon = reconstructImages(XTestCensorFlat, yPred_single, model, mask)
```

3 Question 3: Comparison of Two-Step and Single-Step Inference

Experimental Setup

We compare the two-step (lecture) and single-step (ours) inference methods across five censoring levels: 51%, 32%, 18%, 8%, and 2% missing pixels, obtained by zeroing out the central square $X[:, a:b, a:b]$ for the values specified in the assignment. For each level we report classification accuracy on the MNIST test images and wall-clock inference time measured using `time.perf_counter`.

Classification Accuracy and Inference Time

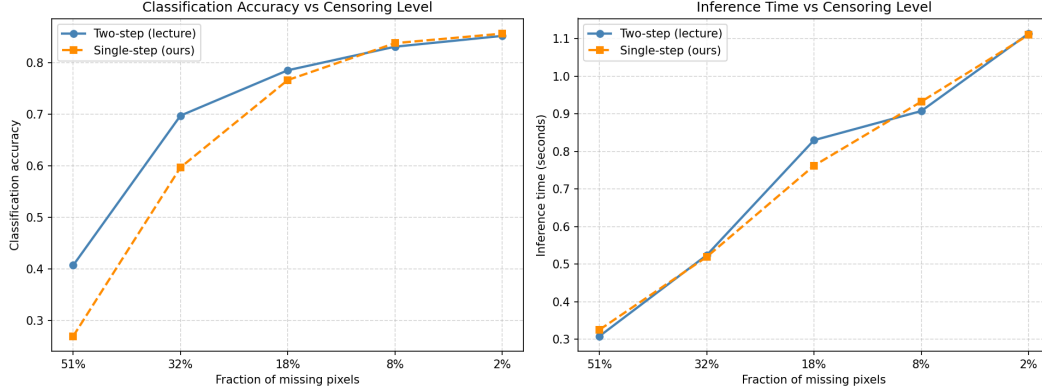


Figure 1: Left: classification accuracy of the two-step and single-step methods as a function of the fraction of missing pixels. Right: wall-clock inference time (seconds) for the same experiment.

Table 1: Classification accuracy and inference time at each censoring level.

Missing pixels	Acc (Two-step)	Acc (Single-step)	Time Two (s)	Time Single (s)
51%	0.4069	0.2693	0.31	0.33
32%	0.6976	0.5970	0.52	0.52
18%	0.7857	0.7667	0.83	0.76
8%	0.8315	0.8384	0.91	0.93
2%	0.8525	0.8569	1.11	1.11

Accuracy. As expected, classification accuracy for both methods decreases as censoring becomes more aggressive, since less information is available from the observed region x^o . As seen in Figure 1 and Table 1, the two-step method substantially outperforms the single-step method at high censoring levels: at 51% missing pixels the gap is $0.41 - 0.27 = 0.14$, and at 32% it is $0.70 - 0.60 = 0.10$. This gap narrows as censoring becomes less aggressive, and the methods perform nearly identically at 8% and 2% missing pixels, where the single-step method is marginally better (0.8384 vs 0.8315 at 8%, and 0.8569 vs 0.8525 at 2%).

This behaviour is explained by the structure of the single-step score. From Question 1, the single-step method predicts

$$\hat{y}_{\text{single}} = \arg \max_c \frac{\mathbb{P}[y = c \mid x^o]}{\sqrt{|\bar{\Sigma}^c|}},$$

penalising each class c by the log-volume $\frac{1}{2} \log |\bar{\Sigma}^c|$ of its conditional missing-pixel distribution. When the missing region is large (aggressive censoring), $\bar{\Sigma}^c$ is a high-dimensional matrix and its determinant varies enormously across classes, causing the penalty to dominate and override the correct posterior signal. When the missing region is small (mild censoring), $|\bar{\Sigma}^c|$ is nearly constant across classes and the penalty has negligible effect, making the two methods essentially equivalent.

The slight edge of the single-step method at 2%–8% is within noise and does not indicate a systematic advantage.

Inference time. Both methods show increasing inference time as censoring becomes *less* aggressive (fewer missing pixels), as seen in the right panel of Figure 1. This is because the observed block Σ_{oo}^c grows larger as more pixels are observed, making the pseudoinverse and the log-determinant computation in Term 1 more expensive for both methods. The single-step method incurs an additional cost from computing the Schur complement $\bar{\Sigma}^c = \Sigma_{mm}^c - \Sigma_{mo}^c (\Sigma_{oo}^c)^{-1} \Sigma_{om}^c$ and its log-determinant for each of the $C = 10$ classes. However, as censoring decreases, the missing block Σ_{mm}^c shrinks, partially offsetting this overhead. Overall the two methods have nearly identical inference times at all censoring levels, differing by at most 0.07 seconds.

Reconstruction Quality

At 2%–8% censoring, the two methods produce nearly identical reconstructions on almost every test image, consistent with the near-identical accuracy scores at these levels. The observed region already strongly constrains the digit shape, so the conditional mean $\hat{x}^m = \bar{\mu}_{\hat{y}}^c$ provides a reasonable infill regardless of which method predicted $\hat{y}$.

As censoring becomes more aggressive (18%–51%), the central region of each digit is entirely removed. At these levels the single-step method makes substantially more classification errors, and because reconstruction is conditioned on the predicted label $\hat{y}$, a wrong prediction causes the infilled region to resemble the wrong digit class entirely, producing implausible completions. The two-step method, having higher classification accuracy at aggressive censoring levels, tends to infill the missing region with a more representative digit shape.

At 51% censoring, the single-step method shows a systematic bias toward classes with compact stroke distributions such as ‘0’ and ‘3’, since these classes have a small $|\bar{\Sigma}^c|$ and thus receive a lower penalty from the $-\frac{1}{2} \log |\bar{\Sigma}^c|$ term; this is directly visible in the reconstructed images where many different true digits are all reconstructed as ‘0’ or ‘3’. The accuracy gap of $0.41 - 0.27 = 0.14$ implies that the two methods disagree on at least 14% of test images; for those images, reconstruction quality differs substantially since each method infills the missing region using a different class conditional mean.

In both methods, all reconstructions appear blurry because the conditional mean of a Gaussian averages over all valid completions consistent with the observed pixels; this blurring is more pronounced when the missing region is large.